Field Hours

CSCE 605 Compiler Design

Original notes for this site. Same topics as the catalog course. Not a reprint of another university's lectures, and not academic credit.

Lecture 1 · 90 minutes

Longest match in the scanner

You scan source by running one automaton over the characters, and you keep the longest match. When the keyword if accepts and an identifier also accepts, you emit the longer lexeme, and equal lengths take the earlier rule. On the input ifx = 12 the identifier wins at length 3, so you do not emit the keyword if of length 2.

You code the scan as a loop that reads one character and updates a state number. A small language needs about 6 kinds: keyword, identifier, integer, operator, punctuation, and whitespace you drop. When the next character has no edge, you emit the last accepting state and put that one character back, which is how 12 and a following semicolon stay two tokens.

One character at a time. The last accept, not the current state, is what you emit.

Read one character -> Update one state -> Record last accept -> Emit and back up

Scan ifx = 12

1. From the start state, i then f then x stay in the identifier. The accept length is 3, which beats the keyword if at length 2.

2. The space has no identifier edge. You emit one identifier, ifx, and you put the space back for a fresh scan.

3. You skip that 1 space. Then = is an operator and 12 is an integer. You emit four tokens, and the keyword rule never fires.

Problem 1. The input is ifx. How many tokens do you emit, and of what kind?

Answer. One identifier of length 3. Longest match will not peel off the keyword if, because the third character still extends the lexeme.

Problem 2. Why do you put one character back when the integer rule stops on 12;?

Answer. The semicolon has no edge out of the integer state. You emit 12 from the last accept, then rescan the semicolon as punctuation. Dropping it hides the separator from the parser.

Lecture 2 · 90 minutes

One token of lookahead

You parse by writing one procedure per nonterminal and choosing a production from a single lookahead token. The mechanism is the FIRST set: a token may start a production only if it belongs to that set. The grammar is LL(1) when the FIRST sets of a nonterminal's alternatives are disjoint, so one token selects at most one

production. If two stmt alternatives both start with id, you must left-factor before the parser can decide.

You consult FOLLOW when a production can derive the empty string. With one lookahead token, you take the empty alternative only if that token sits in FOLLOW of the nonterminal. An expression with + and * needs 2 precedence levels, which you encode as term and factor rather than one left-recursive expr rule. Left recursion would call the same procedure forever, before it consumes a single token.

Predictive descent uses one token because the FIRST sets do not overlap.

1. Compute each FIRST set

2. Require disjoint alternatives

3. Predict from one token

4. Expand that production

Choose between assignment and if

1. stmt is either id = expr ; or if (expr) stmt. FIRST of the first alternative is {id}. FIRST of the second is {if}.

2. Lookahead id selects the assignment. You consume id, then =, then an expr, then one semicolon. You never try the if production.

3. Lookahead if selects the other production with that same one token. If both FIRST sets had contained id, the grammar would not be LL(1).

Problem 1. Both productions for stmt start with id. Can you choose with one lookahead token?

Answer. No. Both FIRST sets contain id, so the grammar is not LL(1). You left-factor, pulling the common id out, before one token can decide.

Problem 2. An optional else derives empty. When may you take that empty production?

Answer. Only when the single lookahead token is in FOLLOW of the statement. If it is not in FOLLOW, you must parse the else instead of stopping.

Lecture 3 · 90 minutes

Shift, reduce, and conflicts

A parser generator builds a shift-reduce table from your productions. Each state holds items that remember how far along a right-hand side you are. Shift pushes the next state. Reduce pops one state per symbol: a rule of 3 symbols pops 3 states, then goto pushes the state for the left-hand side. You do not hand-code that pop count once the table exists.

You read conflicts from the same table. A shift/reduce conflict on + means one item wants to shift + and another wants to reduce. You close an operator conflict by assigning precedence and associativity. Left associativity of + reduces on a second +, so 1+2+3 becomes (1+2)+3 rather than (1+(2+3)). If you leave the conflict, the table is not a deterministic parser.

The same stack, two table actions. The pop count is the length of the right-hand side.

Shift +

Push the state for +

Wait for the next operand

Right associativity does this

Reduce +

Pop the E + E rule

Goto on E

Left associativity does this

Parse 1+2+3 left associative

1. You shift 1, shift +, and shift 2. The lookahead is now the second +.

2. Left associativity reduces E + E before shifting that second +. The stack holds one E, and you have popped 3 states.

3. You then shift +, shift 3, and reduce again. The only tree is (1+2)+3. Right associativity would have shifted the second + instead.

Problem 1. A production has 3 symbols on its right-hand side. How many states does reduce pop?

Answer. 3. Reduce pops one state per symbol, then goto pushes one state for the left-hand side.

Problem 2. The table conflicts on +, and you mark + left associative. What tree do you build for 1+2+3?

Answer. (1+2)+3. Left associativity reduces the first plus when the second plus is the lookahead, instead of shifting it.

Lecture 4 · 90 minutes

Scopes and stack frames

You keep one symbol table per scope and link it to the enclosing scope. Lookup walks that chain and stops at the first hit, so an inner name shadows an outer name with the same spelling. In a procedure with 4 integer locals of 4 bytes, you assign offsets 0, 4, 8, and 12. The frame is 16 bytes before you add slots for saved registers.

You place locals in the stack frame so 2 recursive calls get 2 frames. The call sequence you emit saves a return address and the frame pointer, then subtracts the frame size from the stack pointer. Parameters sit at fixed offsets from that pointer; a call with 2 arguments uses 2 slots and one sign convention. A static link stored in the frame lets a nested procedure read the enclosing activation.

One activation. Offsets are measured from the frame pointer you just saved.

Return address

Saved frame pointer

Four locals, 4 bytes each

Static link if nested

Four integer locals

1. Locals a, b, c, and d are 4 bytes each. Their offsets are 0, 4, 8, and 12.

2. Ignoring saved registers, the frame is 16 bytes. The prologue subtracts 16 from the stack pointer.

3. A nested call stores a static link to this frame. A use of a loads offset 0 through that link, not from the inner frame.

Problem 1. You add a fifth integer local of 4 bytes. What is its offset, and what is the frame size, still ignoring saved registers?

Answer. The offset is 16. The frame size becomes 20. Each local keeps the same 4-byte stride.

Problem 2. Two recursive calls of this procedure are active. How many copies of local a exist?

Answer. Two. Each activation has its own frame, and a sits at offset 0 in that frame. One global slot would make the calls share a single a.

Lecture 5 · 90 minutes

Three-address code and blocks

You lower expressions to three-address instructions, each with one operator and at most 3 operands, such as t1 = a + b. Later passes get easier because you cut this list into basic blocks: straight-line runs with no entry or exit in the middle. A leader starts each block. Leaders are the first instruction, any jump target, and the instruction that sits immediately after a jump.

You give each assignment its own temporary so a definition names one value. In a block of 5 instructions for (a+b)*(a+b), a naive lowering recomputes the sum and uses 3 temporaries, while reusing the sum uses 2. The block ends at a conditional jump, which has 2 successors in the control-flow graph. Dataflow walks that graph. It does not walk source line numbers.

Leaders cut the instruction list. A jump ends a block and names a successor.

1. Mark the leaders

2. Cut basic blocks

3. Link each successor

Four instructions, three leaders

1. The body is: 1, t1 = a + b; 2, if t1 goto 4; 3, t2 = 1; 4, return t1. Instruction 1 is a leader because it is first.

2. Instruction 4 is a leader because it is a jump target. Instruction 3 is a leader because it follows a jump. That is 3 leaders.

3. The blocks are {1, 2}, {3}, and {4}. The jump at 2 has 2 successors: block 3 when the test fails, and block 4 when it holds.

Problem 1. Using that same four-instruction body, which instruction numbers are leaders?

Answer. 1, 3, and 4. Instruction 2 is not a leader. It sits inside the first block; it is the jump, not a target and not an instruction after a jump.

Problem 2. Inside one block you lower (a+b)*(a+b). How many temporaries does a naive lowering use, and how many if you reuse the sum?

Answer. The naive lowering uses 3 temporaries. Reusing the sum uses 2.

Lecture 6 · 90 minutes

Reaching definitions and liveness

You compute reaching definitions with a forward pass on the block graph. A definition of x reaches a use when some path carries it and no later definition of x kills it. The mechanism is a bit vector, one bit per definition. With 8 definitions, each block stores an 8-bit IN set and an 8-bit OUT set. You iterate until a pass changes nothing. The equations are monotone, so that fixed point exists.

Liveness runs backward. A name is live where some path uses it before the next definition. You start from uses at the end of a block and walk predecessors, deleting a temporary that is live on 0 paths. A loop of 3 blocks can take 2 or 3 iterations before the live set stops growing, because the back edge feeds the header. You must not stop after a single pass.

Same graph, opposite directions. One bit per definition, or one bit per name.

Reaching, forward

Union of predecessor OUT

Kill definitions of x

Gen the local definition

Liveness, backward

Union of successor IN

A use makes a name live

A definition clears it

Two definitions reach one use

1. The function has 8 definitions in its bit vector. Block A defines x as d1. Block B defines x as d2. Both flow to block C, which uses x.

2. OUT of A contains d1 and not d2, because defining x kills other definitions of x. OUT of B contains d2 and not d1.

3. IN of C is the union, so both bits are set. The use does not have a unique reaching definition, and you may not treat x as a constant.

Problem 1. A pass changes one block's OUT set among the 8 bits. Do you stop the iteration?

Answer. No. You stop only when a full pass changes nothing. A changed OUT can alter a successor IN on the next pass.

Problem 2. A temporary is live on 0 paths at its definition. What may you do with that instruction?

Answer. Delete it. Nothing uses the value before the next definition. On a loop you confirm the 0 only after the live set has stopped growing.

Lecture 7 · 90 minutes

Folding, copies, and coloring

You fold an instruction whose operands are constants before you allocate registers. If t1 = 2 + 3, you replace later uses of t1 by 5 and you delete the add once t1 is dead. Copy propagation does the same for t2 = t1: uses of t2 become uses of t1, and the copy often disappears. Both rewrites need a single reaching definition. If 2 definitions of t1 reach the use, you do not fold.

You allocate registers from an interference graph. Two temporaries interfere when one is live at the definition of the other. A graph you color with 4 colors fits in 4 registers. A node of degree 5 or more, on that 4-register machine, is a spill: you store it in a frame slot and reload it at each use. Spills add uses, so you recompute liveness once after you insert the loads.

Fold only a unique definition. Spill only after coloring fails, then rebuild liveness.

Fold a constant -> Delete a dead op -> Build interference -> Color or spill

Fold 2 + 3, then color

1. The only definition of t1 that reaches the use is t1 = 2 + 3. You replace the use by 5.

2. t1 is now live on 0 paths, so you delete the add. The block lost 1 instruction.

3. Suppose a remaining temporary has degree 5 and you have 4 registers. You spill that name to the frame and recompute liveness, because each reload is a new use.

Problem 1. Two definitions reach a use of t: t = 2 + 3 and t = a + b. May you rewrite the use to 5?

Answer. No. Folding needs one reaching definition. Two definitions reach the use, so the value is not always 5.

Problem 2. You have 4 registers, and one temporary has interference degree 5. What do you do next?

Answer. Spill it to a frame slot and reload it at each use. Then recompute liveness, because the new loads are uses the old set did not contain.

Lecture 8 · 90 minutes

A compiler you can defend

You can now carry a small source file from characters to optimized three-address code. You scan with longest match, parse with 1 lookahead token or a generated shift-reduce table, and assign frame offsets in a nested symbol table. You then split instructions into basic blocks. On a function with 4 locals of 4 bytes you can name each offset, and you can say which definitions reach a given use.

You can also change that code and defend the change. Constant folding is legal only when one reaching definition supplies the constants, and coloring is legal only for the liveness you computed. Fold 2 + 3 into 5 and, once that temporary is dead, remove 1 instruction. If 4 registers are not enough, spill 1 name and recompute liveness, because each new load is a use the old set missed.

The passes in the order you can now justify, not as a list of tool names.

Longest-match scan

Predict or shift-reduce

Frames, then basic blocks

Dataflow, then folding

Fold, and the frame is unchanged

1. A use of t1 has the single definition t1 = 2 + 3. You fold the use to 5 and delete the add because t1 is dead. That removes 1 instruction.

2. The procedure still has 4 integer locals. Their frame is still 16 bytes. Folding a temporary did not move local offsets.

3. If coloring then needs a fifth register, you spill 1 name into that same frame and you recompute liveness. The fold and the spill are separate edits.

Problem 1. A use of t has two reaching definitions, t = 2 + 3 and t = a + b, and 4 registers are free. May you fold the use to 5?

Answer. No. Folding requires a unique reaching definition, and two definitions reach this use. Free registers do not make the value a constant. Coloring and folding answer different questions.

Problem 2. A procedure has 3 integer locals of 4 bytes. Its body is the four-instruction example whose leaders are 1, 3, and 4. What is the frame size, ignoring saved registers, and how many basic blocks are there?

Answer. The frame is 12 bytes. There are 3 leaders, so there are 3 basic blocks. The frame size does not change the leader rule, and the jumps do not change the 4-byte stride.